



Docker Compose

Ce tutoriel vise à présenter les concepts fondamentaux de Docker Compose en vous guidant dans le développement d'une application Web Python de base.

En utilisant le framework Flask, l'application dispose d'un compteur de visites dans Redis, fournissant un exemple pratique de la manière dont Docker Compose peut être appliqué dans des scénarios de développement Web.

Il s'agit d'un exemple non normatif qui met simplement en évidence les éléments clés que vous pouvez faire avec Compose.

Prérequis

Assurez-vous d'avoir :

- Installé la dernière version de Docker Compose
- Une compréhension de base des concepts Docker et de son fonctionnement

Étape 0 : C'est quoi Docker Compose ?

Commencer par regarder cette [Vidéo introductive à Docker Compose](#).

Étape 1 : Configuration

1. Créer un répertoire pour le projet :

```
$ mkdir composetest  
$ cd composetest
```

2. Créez un fichier appelé `app.py` dans le répertoire de votre projet et collez-y le code suivant :



```
import time
import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
    retries = 5
    while True:
        try:
            return cache.incr('hits')
        except redis.exceptions.ConnectionError as exc:
            if retries == 0:
                raise exc
            retries -= 1
            time.sleep(0.5)

@app.route('/')
def hello():
    count = get_hit_count()
    return f'Hello World! I have been seen {count} times.\n'
```

Dans cet exemple, le nom d'hôte `redis` du conteneur Redis sur le réseau de l'application et le port par défaut `6379` sont utilisés.

Note : Notez la manière dont la fonction `get_hit_count` est écrite. Cette boucle de nouvelle tentative de base tente la requête plusieurs fois si le service Redis n'est pas disponible. Cela est utile au démarrage lorsque l'application est en ligne, mais rend également l'application plus résiliente si le service Redis doit être redémarré à tout moment pendant la durée de vie de l'application. Dans un cluster, cela permet également de gérer les pertes de connexion momentanées entre les nœuds.

3. Créez un autre fichier appelé `requirements.txt` dans le répertoire de votre projet et collez-y le code suivant :

```
flask
redis
```

4. Créez un `Dockerfile` et collez le code suivant :



```
# syntax=docker/dockerfile:1
FROM python:3.10-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run", "--debug"]
```

Ce que fait le Dockerfile

- Créer une image en commençant par l'image Python 3.10.
- Définir le répertoire de travail sur `/code`.
- Définir les variables d'environnement utilisées par la commande `flask`.
- Installer gcc et d'autres dépendances
- Copier le fichier `requirements.txt` et installer les dépendances Python.
- Ajouter des métadonnées à l'image pour décrire que le conteneur écoute sur le port 5000.
- Copier le répertoire actuel `.` du projet dans le répertoire de travail `.` de l'image.
- Définir la commande par défaut du conteneur sur `flask run --debug`.

Étape 2 : définir les services dans un fichier Compose

Compose simplifie le contrôle de l'ensemble de votre pile d'applications, facilitant la gestion des services, des réseaux et des volumes dans un seul fichier de configuration YAML compréhensible.

Créez un fichier appelé `compose.yaml` dans le répertoire de votre projet et collez ce qui suit :

```
services:
  web:
    build: .
```



```
ports:
  - "8000:5000"
redis:
  image: "redis:alpine"
```

Ce fichier **Compose** définit deux services : `web` et `redis`.

Le `web` service utilise une image créée à partir du répertoire de `Dockerfile` actuel. Il lie ensuite le conteneur et la machine hôte au port exposé, `8000`. Cet exemple de service utilise le port par défaut du serveur Web Flask, `5000`.

Le service `redis` utilise une image [Redis](#) publique extraite du registre Docker Hub.

Étape 3 : créer et exécuter votre application avec Compose

Avec une seule commande, vous créez et démarrez tous les services de votre fichier de configuration.

1. Depuis votre répertoire de projet, démarrez votre application en exécutant `docker compose up`.

```
$ docker compose up

Creating network "composetest_default" with the default driver
Creating composetest_web_1 ...
Creating composetest_redis_1 ...
Creating composetest_web_1
Creating composetest_redis_1 ... done
Attaching to composetest_web_1, composetest_redis_1
web_1      | * Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
redis_1    | 1:C 17 Aug 22:11:10.480 # o000o000o000o Redis is starting o000o000o000o
redis_1    | 1:C 17 Aug 22:11:10.480 # Redis version=4.0.1, bits=64, commit=00000000, modified=0, pid=1
redis_1    | 1:C 17 Aug 22:11:10.480 # Warning: no config file specified, using the default config. In
web_1      | * Restarting with stat
redis_1    | 1:M 17 Aug 22:11:10.483 * Running mode=standalone, port=6379.
redis_1    | 1:M 17 Aug 22:11:10.483 # WARNING: The TCP backlog setting of 511 cannot be enforced becau
web_1      | * Debugger is active!
redis_1    | 1:M 17 Aug 22:11:10.483 # Server initialized
redis_1    | 1:M 17 Aug 22:11:10.483 # WARNING you have Transparent Huge Pages (THP) support enabled in
web_1      | * Debugger PIN: 330-787-903
redis_1    | 1:M 17 Aug 22:11:10.483 * Ready to accept connections
```



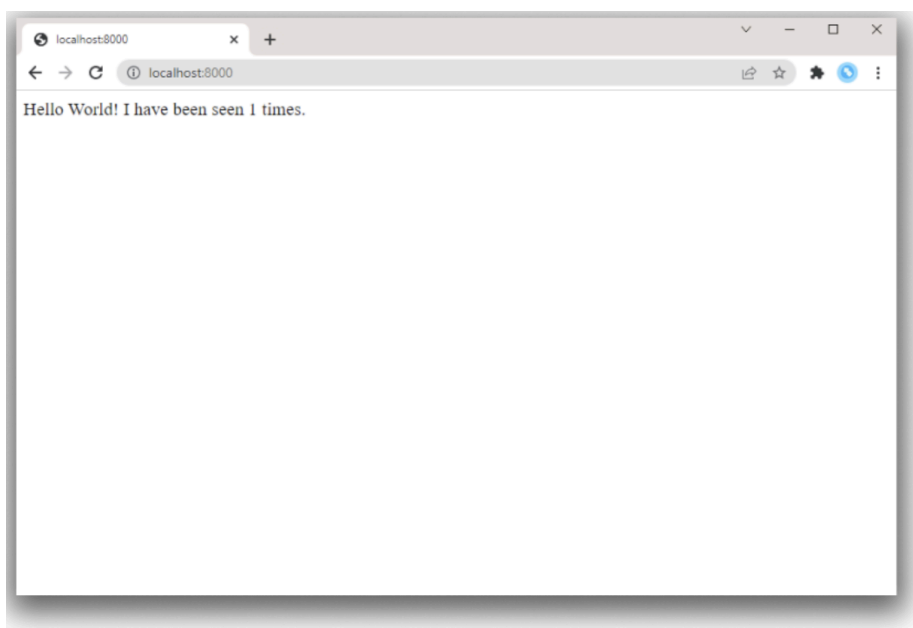
1. Compose extrait une image Redis, crée une image pour votre code et démarre les services que vous avez définis. Dans ce cas, le code est copié de manière statique dans l'image au moment de la création.
2. Entrez <http://localhost:8000/> dans un navigateur pour voir l'application en cours d'exécution.

Si cela ne résout pas le problème, vous pouvez également essayer

<http://127.0.0.1:8000/>.

Vous devriez voir un message dans votre navigateur indiquant :

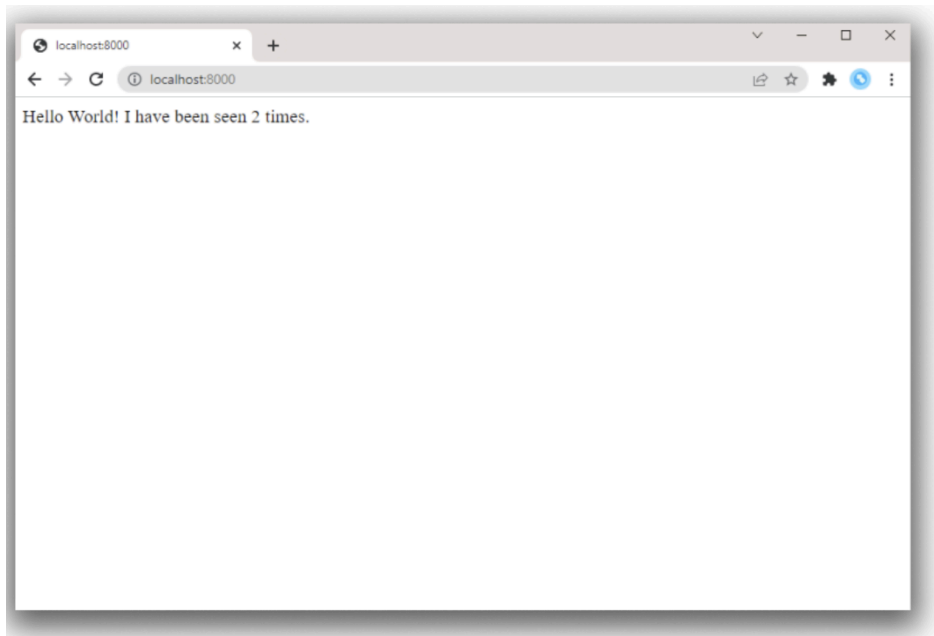
Hello World! I have been seen 1 times.



3. Rafraîchir la page.

Le nombre devrait augmenter.

Hello World! I have been seen 2 times.



4. Passez à une autre fenêtre de votre terminal et tapez `docker image ls` pour répertorier les images locales.

La liste des images à ce stade doit contenir les images `redis` et `web`.

Vous pouvez inspecter les images avec `docker inspect <tag or id>`.

5. Arrêtez l'application, soit en l'exécutant `docker compose down` depuis votre répertoire de projet dans le deuxième terminal, soit en appuyant `CTRL+C` sur le terminal d'origine où vous avez démarré l'application.

Étape 4 : Modifiez le fichier Compose pour utiliser Compose Watch

Modifiez le fichier `compose.yaml` dans votre répertoire de projet pour l'utiliser `watch` afin de pouvoir prévisualiser vos services Compose en cours d'exécution qui sont automatiquement mis à jour lorsque vous modifiez et enregistrez votre code :

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
  develop:
    watch:
```



```
- action: sync
  path: .
  target: /code
redis:
  image: "redis:alpine"
```

Chaque fois qu'un fichier est modifié, Compose synchronise le fichier avec l'emplacement correspondant à `/code` l'intérieur du conteneur. Une fois copié, le bundler met à jour l'application en cours d'exécution sans redémarrage.

Pour plus d'informations sur le fonctionnement de Compose Watch, consultez [Utiliser Compose Watch](#). Vous pouvez également consulter [Gérer les données dans des conteneurs](#) pour découvrir d'autres options.

Note : Pour que cet exemple fonctionne, l'option `--debug` est ajoutée au fichier `Dockerfile`. L'option `--debug` dans Flask permet le rechargement automatique du code, ce qui permet de travailler sur l'API backend sans avoir à redémarrer ou reconstruire le conteneur. Après avoir modifié le fichier `.py`, les appels d'API suivants utiliseront le nouveau code, mais l'interface utilisateur du navigateur ne s'actualisera pas automatiquement dans ce petit exemple. La plupart des serveurs de développement frontend incluent une prise en charge native du rechargement en direct qui fonctionne avec Compose.

Étape 5 : reconstruisez et exécutez l'application avec Compose

Depuis votre répertoire de projet, tapez `docker compose watch` ou `docker compose up --watch` pour créer et lancer l'application et démarrer le mode de surveillance des fichiers.

```
$ docker compose watch
[+] Running 2/2
✓ Container docs-redis-1 Created
0.0s
✓ Container docs-web-1      Recreated
0.1s
Attaching to redis-1, web-1
  ● watch enabled
...
```



Vérifiez à nouveau le message dans un navigateur Web et actualisez-le pour voir l'incrément du compte.

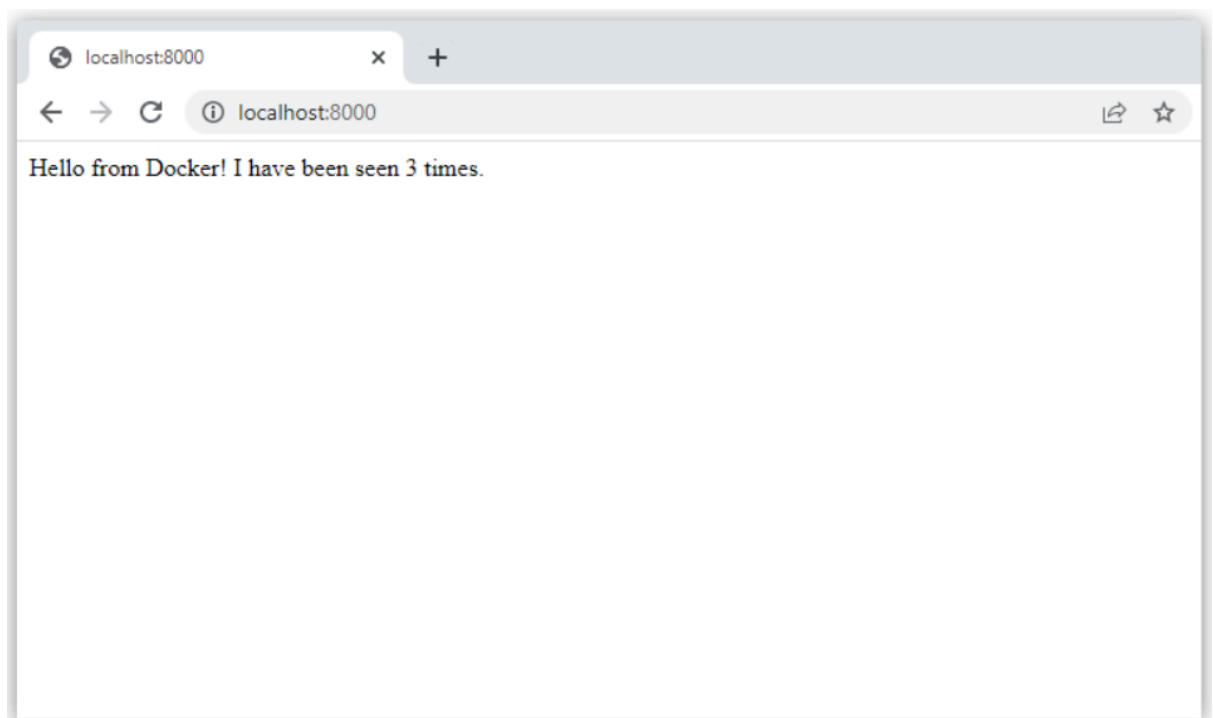
Étape 6 : Mettre à jour l'application

Pour voir Compose Watch en action :

1. Modifiez le message d'accueil `app.py` et enregistrez-le. Par exemple, modifiez le message `Hello World!` en `Hello from Docker!`:

```
return f'Hello from Docker! I have been seen {count} times.\n'
```

2. Actualisez l'application dans votre navigateur. Le message d'accueil devrait être mis à jour et le compteur devrait continuer à augmenter.



3. Une fois que vous avez terminé, exécutez `docker compose down`.

Étape 7 : Répartissez vos services

L'utilisation de plusieurs fichiers Compose vous permet de personnaliser une application Compose pour différents environnements ou flux de travail. Cela est utile pour les applications volumineuses qui peuvent utiliser des dizaines de conteneurs, avec une propriété répartie entre plusieurs équipes.



1. Dans votre dossier de projet, créez un nouveau fichier Compose appelé `infra.yaml`.
2. Coupez le service Redis de votre fichier `compose.yaml` et collez-le dans votre nouveau fichier `infra.yaml`. Assurez-vous d'ajouter l'attribut `services` de niveau supérieur en haut de votre fichier. Votre fichier `infra.yaml` devrait maintenant ressembler à ceci :

```
services:
  redis:
    image: "redis:alpine"
```

Dans votre fichier `compose.yaml`, ajoutez l'attribut `include` de niveau supérieur ainsi que le chemin d'accès au fichier `infra.yaml`.

```
include:
  - infra.yaml
services:
  web:
    build: .
    ports:
      - "8000:5000"
  develop:
    watch:
      - action: sync
        path: .
        target: /code
```

1. Exécutez `docker compose up` pour créer l'application avec les fichiers Compose mis à jour et exécutez-la. Vous devriez voir le message `Hello world` dans votre navigateur.

Il s'agit d'un exemple simplifié, mais il illustre le principe de base `include` et la manière dont il peut faciliter la modularisation d'applications complexes en sous-fichiers Compose. Pour plus d'informations sur `include` l'utilisation de plusieurs fichiers Compose, voir [Utilisation de plusieurs fichiers Compose](#) .

Étape 8 : Expérimentez avec d'autres commandes

Si vous souhaitez exécuter vos services en arrière-plan, vous pouvez passer l'indicateur `-d` (pour le mode « détaché ») à `docker compose up` et utiliser `docker compose ps` pour voir ce qui est en cours d'exécution :



```
$ docker compose up -d
Starting composetest_redis_1...
Starting composetest_web_1...
```

```
$ docker compose ps
```

Name	Command	State
composetest_redis_1	docker-entrypoint.sh redis ...	Up
composetest_web_1	flask run	Up

Exécutez `docker compose --help` pour voir les autres commandes disponibles.

Si vous avez démarré Compose avec `docker compose up -d`, arrêtez vos services une fois que vous avez terminé avec eux :

```
$ docker compose stop
```

Vous pouvez tout démonter, en retirant entièrement les conteneurs, avec la commande `docker compose down`.

Références

<https://docs.docker.com/compose/gettingstarted/>

<https://github.com/docker/awesome-compose>

[Liste complète des commandes Docker](#)

[Vidéo introductive à Docker Compose](#)